# Puppet Branding Module

RHEL 8 / 9 / 10 - custom OpenVox/Puppet module documentation

**Purpose:** Install a site logo and apply it to the GNOME login screen on graphical RHEL systems.

## Module layout

```
modules/custom/
  manifests/branding.pp
  manifests/dconf.pp
  functions/source_candidates.pp
  templates/override-logo.epp
  files/branding/README.md
```

## Configuration

```
include custom::branding

custom::branding::manage_gui: true
custom::branding::logo_source: puppet:///modules/custom/branding/corp-logo.png
custom::branding::logo_path: /usr/local/share/corp/corp-logo.png
```

**Behavior and safety:** The custom_gui fact gates graphical changes. The logo source supports network node (certname), network project, network role, and network default overrides, with optional OS-specific files at each level. Puppet writes and locks the GDM dconf key, then refreshes the dconf databases.

**Validation status:** This module is part of a STIG hardening automation repository. STIG compliance has not been validated, and the code has not been rigorously tested. Review and test in an isolated environment before production use.

## Branding

modules/custom/manifests/branding.pp

```
# @summary
#   Installs a site logo and applies it to the GNOME login screen.
#
# @param manage_gui
#   Whether to manage graphical branding. Defaults to the custom_gui fact.
# @param logo_source
#   Default Puppet source URI for the common logo image.
# @param logo_path
#   Absolute destination path for the logo on managed systems.
class custom::branding (
  Boolean   $manage_gui  = $facts['custom_gui'] ? {
    true    => true,
    default => false,
  },
  String[1] $logo_source = 'puppet:///modules/custom/branding/corp-logo.png',
  String[1] $logo_path   = '/usr/local/share/corp/corp-logo.png',
) {
  if $facts['os']['family'] != 'RedHat' or $facts['os']['release']['major'] !~ /^(8|9|10)$/ {
    fail('custom::branding supports only RHEL 8, RHEL 9, and RHEL 10')
  }

  if $manage_gui {
    contain custom::dconf

    file { 'custom branding directory':
      ensure => directory,
      path   => '/usr/local/share/corp',
      owner  => 'root',
      group  => 'root',
      mode   => '0755',
    }

    file { 'custom branding logo':
      ensure       => file,
      path         => $logo_path,
      owner        => 'root',
      group        => 'root',
      mode         => '0644',
      source       => custom::source_candidates($logo_source),
      sourceselect => 'first',
      require      => File['custom branding directory'],
    }

    file { 'custom gdm logo configuration':
      ensure  => file,
      path    => '/etc/dconf/db/gdm.d/01-override-logo',
      owner   => 'root',
      group   => 'root',
      mode    => '0644',
      content => epp('custom/override-logo.epp', {
        'logo_path' => $logo_path,
      }),
      require => File['custom branding logo'],
      notify  => Exec['update custom dconf databases'],
    }

    file { 'custom gdm logo lock':
      ensure  => file,
      path    => '/etc/dconf/db/gdm.d/locks/01-override-logo',
      owner   => 'root',
      group   => 'root',
      mode    => '0644',
      content => "# Managed by corp Puppet.\n# Manual changes will be replaced.\n/org/gnome/login-screen/logo\n",
      notify  => Exec['update custom dconf databases'],
    }
  }
}
```

```
# @summary
#   Creates the shared dconf directories and refresh command used by this module.
class custom::dconf {
  file { 'custom dconf directory':
    ensure => directory,
    path   => '/etc/dconf',
  }

  file { 'custom dconf database directory':
    ensure => directory,
    path   => '/etc/dconf/db',
  }

  file { 'custom gdm dconf directory':
    ensure => directory,
    path   => '/etc/dconf/db/gdm.d',
  }

  file { 'custom gdm dconf locks directory':
    ensure => directory,
    path   => '/etc/dconf/db/gdm.d/locks',
  }

  file { 'custom local dconf directory':
    ensure => directory,
    path   => '/etc/dconf/db/local.d',
  }

  file { 'custom local dconf locks directory':
    ensure => directory,
    path   => '/etc/dconf/db/local.d/locks',
  }

  exec { 'update custom dconf databases':
    command     => '/usr/bin/dconf update',
    refreshonly => true,
    require     => [
      File['custom gdm dconf directory'],
      File['custom gdm dconf locks directory'],
      File['custom local dconf directory'],
      File['custom local dconf locks directory'],
    ],
  }
}
```

```
# Returns node, project, role, network default, then shared default candidates.
# Each tier tries the OS major version first, then its version-independent file.
# Pass the base filename without an OS suffix (see NETWORK_OVERRIDES.md).
function custom::source_candidates(String[1] $default_source) >> Array[String] {
  $network = $trusted['extensions']['pp_network']
  $project = $trusted['extensions']['pp_project']
  $role = $trusted['extensions']['pp_role']
  $certname = $trusted['certname']
  $os_major = $facts.dig('os', 'release', 'major')

  if $os_major != undef and $os_major != '' {
    if $os_major !~ String or $os_major !~ /\A[0-9]+\z/ {
      fail('custom::source_candidates OS major version must contain only digits')
    }
  }

  if $network == undef or $network == '' {
    $suffixes = ['']
  } else {
    # Reject path traversal and URI delimiters in filename suffixes.
    [$network, $project, $role, $certname].each |$identity| {
      if $identity != undef and $identity != '' {
        if $identity !~ String or $identity !~ /\A[A-Za-z0-9][A-Za-z0-9_.-]*\z/ {
          fail('custom::source_candidates identities must start with a letter or digit and contain only letters, digits, underscores, dots, or hyphens')
        }
      }
    }
    $node_suffixes = $certname ? {
      undef => [], '' => [], default => [".${network}.${certname}"],
    }
    $project_suffixes = $project ? {
      undef => [], '' => [], default => [".${network}.${project}"],
    }
    $role_suffixes = $role ? {
      undef => [], '' => [], default => [".${network}.${role}"],
    }
    $suffixes = $node_suffixes + $project_suffixes + $role_suffixes + [".${network}", '']
  }

  $suffixes.reduce([]) |$candidates, $suffix| {
    if $os_major == undef or $os_major == '' {
      $candidates + ["${default_source}${suffix}"]
    } else {
      $candidates + ["${default_source}.${os_major}${suffix}", "${default_source}${suffix}"]
    }
  }
}
```

```
<%- | String[1] $logo_path | -%>
# Managed by corp Puppet.
# Manual changes will be replaced.
[org/gnome/login-screen]
logo='<%= $logo_path %>'
```

## Branding

modules/custom/files/branding/README.md

```
# Branding image

Add the production logo as `corp-logo.png` in this directory before applying
`custom::branding` to a graphical node. Do not use a placeholder image: the
managed source must be a valid image that the GDM login screen can render.
```